Cookies in APIs (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p4-cookies-in-flask-api)  (https://github.com/learn-co-curriculum/python-p4-cookies-in-flask-api/issues/new)

Learning Goals

- Configure a Flask API to use cookies.
- Use the developer tools to inspect cookies.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

Since cookies are such an important part of most web applications, Flask has excellent support for cookies and sessions baked in. To test these out, let's make a simple API to display our cookie and session data.

Working With Sessions and Cookies

We've included some starter code for a Flask API application with this lesson so you can see a basic example of working with sessions and cookies. The configuration is already done, so we can work on inspecting sessions and cookies in the app and see how we can interact with them in our code.

To set up and run the Flask application, run:

```
$ pipenv install && pipenv shell
$ python server/app.py
```

Then, in the browser, make a request to `http://localhost:5555/sessions/hello`. This will run the code in our `show_session()` view function:

```
@app.route('/sessions/<string:key>', methods=['GET'])
def show_session(key):

    session["hello"] = session.get("hello") or "World"
    session["goodnight"] = session.get("goodnight") or "Moon"

    response = make_response(jsonify({
        'session': {
            'session_key': key,
            'session_value': session[key],
            'session_accessed': session.accessed,
        },
        'cookies': [{cookie: request.cookies[cookie]}
            for cookie in request.cookies],
    }), 200)

    response.set_cookie('mouse', 'Cookie')
```

```
    return response
```

In this function, we're setting values on the `session` object and the `cookies` object, and serializing them in the response so we can view their values in the browser.

Note that we are using a ternary operator to set values for our session; we typically want our session to stay consistent until a user ends it, so we only set certain values if they do not already exist.

The first time a user makes a request to this API, Flask will include the `Set-Cookie` **response header** with our sessions and cookies values, which will instruct the browser to store these values in memory and send them with any future requests on this domain.

After making the request, you should see something like this in the browser:


```
{
  "cookies": [
    {
      "mouse": "Cookie"
    },
    {
      "session": "eyJnb29kbmlnaHQiOiJNb29uIiwiaGVsbG8iOiJXb3JsZCJ9.Y3KXKQ.oTqGI6rmhKDNLizZaHfJadRybUc"
    }
```

```
  ],
  "session": {
    "session_accessed": true,
    "session_key": "hello",
    "session_value": "World"
  }
}
```

From this, we can see that the session and cookies objects can both be used to store key-value pairs of data. The entire session object is actually stored in that `session` cookie, in a signed and encrypted format, which makes it impossible for users to tamper with.

You can view cookie information directly in the browser as well. In the developer tools, find the **Application** tab, and go to the **Cookies** section (under "Storage" in the pane on the left). There, you'll find all the cookies for our domain (`http://localhost:5555`):

Cookies can be edited directly in the dev tools. Try changing the value of the `mouse` key to something new. Then refresh the page in the browser to make another request. If you try to edit the `session` cookie, on the other hand, it will have no effect thanks to Flask security features like signing and encryption.

Conclusion

Cookies are an integral part of modern web applications; they help keep track of **stateful** information in an inherently **stateless** protocol by automatically passing additional data with each request using the headers. We can get a better sense of how cookies are being used by websites using the browser dev tools.

Check For Understanding

Before you move on, make sure you can answer the following questions:

▶ *1. `session` is a unique object in Flask, but it stores elements in key-value pairs. This makes it similar to what built-in data structure?*

▶ *2. What are the two ways you can inspect a website's cookies using the browser dev tools?*

Resources

- **API - Flask: class flask.session (https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)**
- **Get and set cookies with Flask - pythonbasics.org (https://pythonbasics.org/flask-cookies/)**
- **SameSite Cookies Explained (https://web.dev/samesite-cookies-explained/)**
- **Chrome DevTools: Working With Cookies (https://developer.chrome.com/docs/devtools/storage/cookies/)**

This tool needs to be loaded in a new browser window

Load Cookies in APIs (CodeGrade) in a new window